ICPay Cloud

Bucket Storage Module

Technical Reference and Architecture Document

Backend Canister: 6vbhm-nqaaa-aaaan-q6muq-cai
Live Application: https://icpay.app/bucket
Document Reference: PR #50 — Chunked Uploads
Last Updated: August 2026

Engineered by - www.prasangapokharel.com

1. Module Overview

ICPay Cloud is an encrypted file storage module built directly into the ICPay wallet canister. The module allows a user to complete the following actions:

- Pay ICP from an ICPay balance to create a bucket, which is a named storage container.
- Upload files such as images, documents, source code, and archives. Video files are not permitted.
- Share public files through CDN style URLs, either the raw.icp0.io domain or an optional CDN proxy.
- Renew buckets every thirty days. Time stacks onto the remaining period when a bucket is renewed early.

All funds and files represent real, on-chain data. The frontend is a static export hosted on Vercel, while the bucket logic is executed by a Motoko canister on the Internet Computer.

2. Architecture Overview

2.1 Backend Architecture — Four Layers

Every request flows downward through the stack in strict order. No layer may be skipped.

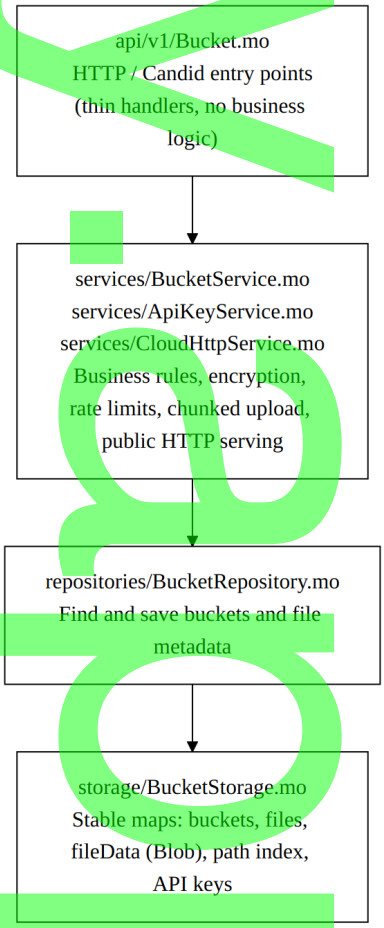

Figure 1 — Backend canister layer flow (strict top-to-bottom order)

Supporting Modules

The following modules support the layered stack but do not sit inside it directly.

Module	Role
config/Config.mo	Limits, pricing constants, rate limits
utils/FileValidator.mo	Extension allow / block list, magic-byte checks
utils/BlobUtil.mo	Join and assemble Blob chunks (Blob preferred over [Nat8])
utils/BucketCrypto.mo	Per-bucket encryption at rest
utils/BucketUrls.mo	Public CDN and raw URL construction
types.mo	Bucket, StoredFile, FileUploadSession, and related types

The canister is defined as a persistent actor within main.mo. Storage maps survive canister upgrades through orthogonal persistence.

2.2 Frontend Architecture — Three Layers

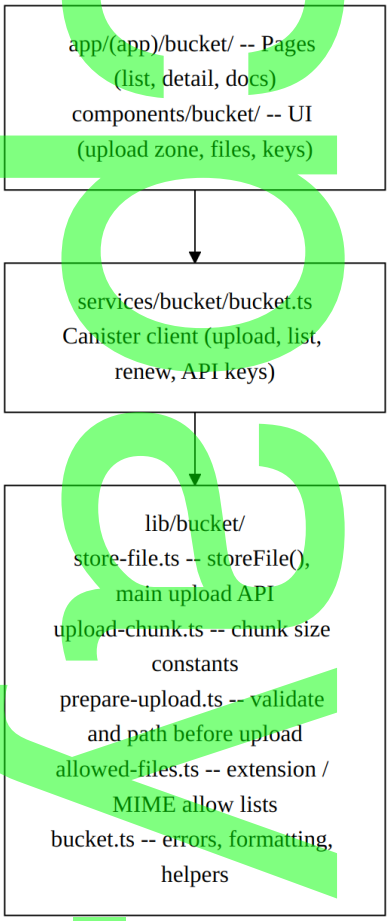

Figure 2 — Frontend layer flow, pages through service client to pure helpers

3. Size Limits — Protocol vs. Product

The constants below represent two distinct categories of limitation. The Internet Computer protocol limit cannot be altered through configuration, while the product limit is an ICPay business decision that may be changed with a canister deploy.

Constant	Value	Meaning
IC_INGRESS_MAX_BYTES	2,097,152 (2 MiB)	Internet Computer protocol hard cap per update message. Cannot be changed in configuration.
BUCKET_UPLOAD_CHUNK_BYTES	700,000 (~700 KB)	Size of each chunk in a chunked upload. Remains safely under 2 MiB including Candid overhead.
BUCKET_UPLOAD_SINGLE_MAX	700,000	Maximum size for the direct uploadFile() call used by API and scripts. The UI always chunks.

Constant	Value	Meaning
BUCKET_MAX_FILE_BYTES	10,000,000 (10 MB)	ICPay product cap. Maximum size of an assembled file after all chunks are joined.
HTTP_MAX_BODY_BYTES	1,900,000	Maximum bytes per HTTP response chunk when serving files through http_request.

Chunking exists because a five megabyte file sent in a single call produces a Candid message of roughly three to five megabytes, which the Internet Computer rejects with an HTTP 413 response before the canister executes any code. Chunking splits the file into a series of small update calls, which the canister then reassembles.

Stable memory on the Internet Computer theoretically allows up to approximately 500 GiB per canister. ICPay intentionally caps uploads at 10 MB per file for reasons of billing and performance.

4. Upload Flow — Chunked (Default Path)

This sequence is used by both the user interface and the storeFile() helper for every upload.

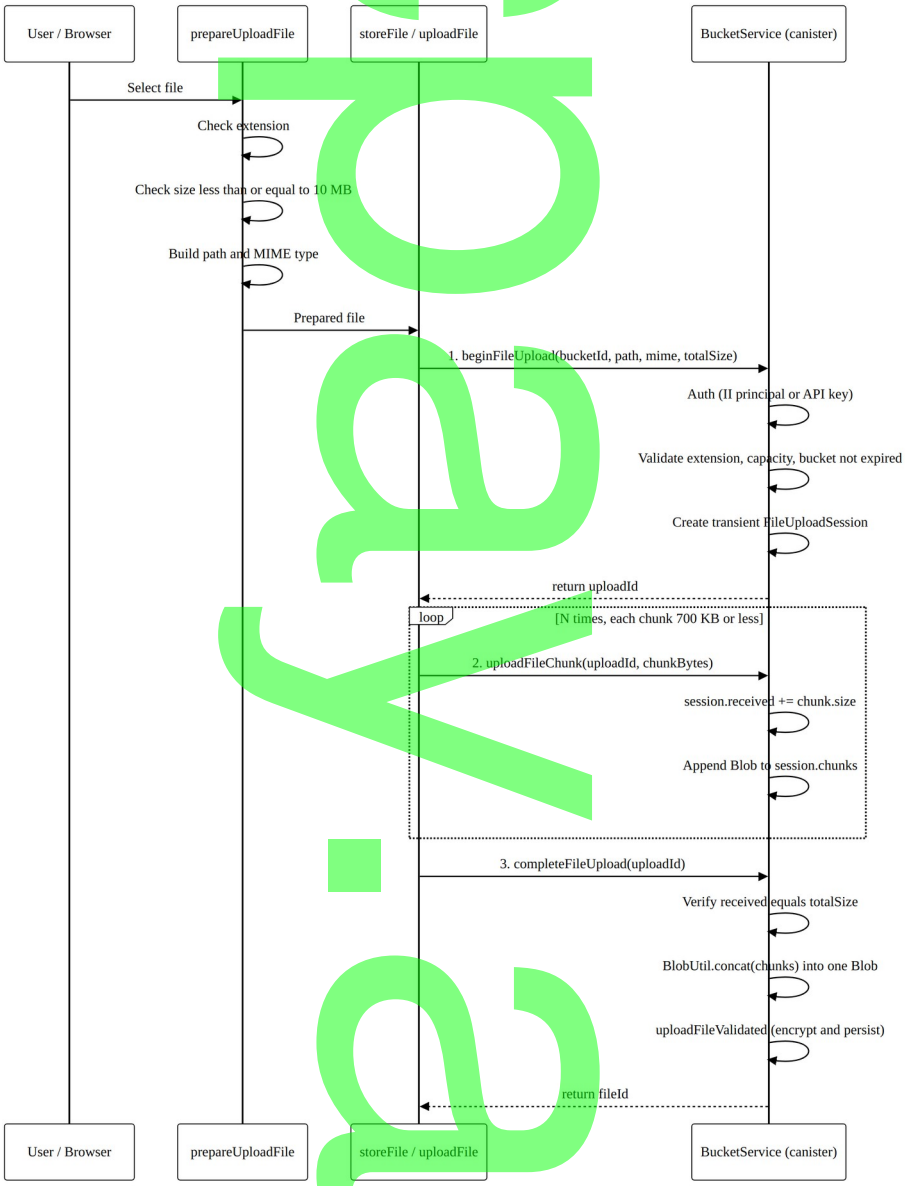

Figure 3 — Chunked upload sequence from browser to backend canister

Session Lifetime and Rate Limiting

- Upload sessions are transient and are lost on a canister upgrade.
- Stale sessions are purged after thirty minutes, governed by BUCKET_UPLOAD_SESSION_TTL_NS.
- The upload rate limit is forty upload calls per minute per principal, governed by RATE_BUCKET_UPLOAD.

- A full ten megabyte file uses approximately seventeen calls in total: one begin call, roughly fifteen chunk calls, and one complete call.

5. Validation Layers

The table below lists every point at which an uploaded file is checked, in the order those checks occur.

Step	Location	What Is Checked
IC ingress	Internet Computer replica	Entire update message is 2 MiB or less, checked before the canister runs
File picker	frontend/lib/bucket/allowed-files.ts	Extension allowed; video files blocked
Prepare	frontend/lib/bucket/prepare-upload.ts	Same checks as above, plus size 10 MB or less
Begin upload	BucketService.beginFileUpload	Path, extension, total size, authentication, capacity, bucket active
Each chunk	BucketService.uploadFileChunk	Chunk non-empty, 700 KB or less, cumulative size within declared total
Complete	BucketService.uploadFileValidated	Full blob: magic bytes, encryption, persistence
Single upload	BucketService.uploadFile	Blob is 700 KB or less only; legacy and API path

Over eighty file extensions are permitted, spanning images, documents, code, archives, audio, and fonts. Video is blocked at both the extension level and the magic-byte level.

6. After Upload — Encryption and Storage

Within the uploadFileValidated() function, the following sequence takes place.

- Normalize the content type using FileValidator.normalizeUpload.
- Derive a per-bucket encryption key using BucketCrypto.deriveKey(owner, bucketId).
- Encrypt the plaintext into a ciphertext Blob together with a fingerprint checksum.
- Save metadata in the files map and the ciphertext in the fileData map, both keyed by fileId.
- Index the path as bucketId:path pointing to fileId, enabling fast lookup.
- Update the bucket storageUsed value.

Files are never stored as plaintext on the canister. Decryption occurs only at the point of download.

7. Download and Public URLs

7.1 Authenticated Download

Authenticated downloads are performed through a canister query:

```
downloadFile(identity, bucketId, path)  ->  decrypted Blob
```

This method is used for private buckets or for owner-level access.

7.2 Public CDN (Anonymous HTTP)

Public buckets expose files through the canister's http_request handler, implemented in CloudHttpService.mo:

```
GET https://6vbhm-nqaaa-aaaan-q6muq-cai.raw.icp0.io/cloud/{bucketName}/{path}
```

- The request path is parsed by the BucketHttp utilities.
- Files are decrypted in chunks for large responses, respecting the 2 MiB IC response limit.
- Content-Type is set from the stored file metadata.

The bucket detail page allows a user to toggle between Raw and CDN URL modes and to copy the resulting base URL.

8. API Keys

Each bucket supports up to ten API keys, each with a granular set of permissions.

Permission	Allows
read	List and download files
write	Upload files, including chunked uploads
delete	Delete files

Keys are created from the bucket detail interface using the Keys button. The secret value is shown once at creation time and should be recorded immediately. It is passed as the optional final argument to uploadFile, beginFileUpload, completeFileUpload, or deleteFile.

9. Billing and Lifecycle

The following table summarizes billing terms and the bucket lifecycle.

Concept	Value
Billing period	30 days
Payment method	Deducted from the user's ICPay ICP balance
Pricing basis	Live from canister — cycle cost plus 50% margin, in

Concept	Value
	0.5 ICP steps
Expired bucket	Read-only; renewal is required to upload again
Renewal	Additional time stacks onto the remaining period

9.1 Capacity Tiers and Indicative Pricing

Bucket capacity is selected at creation time from a fixed set of tiers. Actual price is always quoted live from the canister at 0.5 ICP increments and reflects current cycle cost plus a 50% margin; the figures below indicate relative scale across tiers only.

Tier	Capacity	Billing Period	Pricing Basis
1	1 GB	30 days	Live quote, cycle cost + 50%
2	5 GB	30 days	Live quote, cycle cost + 50%
3	10 GB	30 days	Live quote, cycle cost + 50%
4	25 GB	30 days	Live quote, cycle cost + 50%
5	50 GB	30 days	Live quote, cycle cost + 50%
6	100 GB	30 days	Live quote, cycle cost + 50%
7	250 GB	30 days	Live quote, cycle cost + 50%
8	500 GB	30 days	Live quote, cycle cost + 50%

10. Key Files Reference

10.1 Backend

File	Purpose
src/api/v1/Bucket.mo	All bucket Candid endpoints
src/services/BucketService.mo	Core business logic, approximately 1000 lines
src/services/CloudHttpService.mo	Public HTTP file serving
src/services/ApiKeyService.mo	API key CRUD operations
src/repositories/BucketRepository.mo	Data access layer
src/storage/BucketStorage.mo	Stable storage maps
src/config/Config.mo	BUCKET_* constants
src/utils/FileValidator.mo	Type validation
src/utils/BlobUtil.mo	Blob concatenation for chunk assembly
src/utils/BucketCrypto.mo	Encrypt and decrypt routines
testing/bucket/Flow.test.mo	End-to-end tests, including a 2.5 MB chunked PHP upload

10.2 Frontend

File	Purpose
lib/bucket/store-file.ts	storeFile() — primary upload API
lib/bucket/upload-chunk.ts	Chunk constants, kept in sync with Config.mo
lib/bucket/prepare-upload.ts	Pre-upload validation
lib/bucket/allowed-files.ts	Allow and block extension lists
services/bucket/bucket.ts	Canister service wrapper
services/wallet.ts	Candid IDL, including chunk methods
app/(app)/bucket/[id]/bucket-detail.tsx	Bucket detail page, Keys, and Renew
components/bucket/bucket-upload-zone.tsx	Drag-and-drop upload interface

11. TypeScript Usage Example

The example below shows the standard pattern for validating and uploading a file. Chunking is handled automatically inside storeFile().

```
import { prepareUploadFile } from "@/lib/bucket/prepare-upload"
import { storeFile } from "@/lib/bucket/store-file"

// 1. Validate and prepare path and MIME type
```

```
const prepared = await prepareUploadFile(file)

// 2. Upload — chunking is automatic (2 MiB IC ingress limit)
const result = await storeFile(identity, prepared.file, {
  bucketId: "your-bucket-id",
  path: prepared.path,
  contentType: prepared.contentType,
  onProgress: (pct) => console.log(`${pct}%`),
})

if ("err" in result) {
  throw new Error(result.err)
}

console.log("Uploaded file id:", result.ok)
```

The equivalent call using the service layer directly is shown below. Both approaches call the same chunked path internally.

```
import { uploadFile } from "@/services/bucket/bucket"

await uploadFile(identity, bucketId, path, file, onProgress, contentType)
```

12. Candid Endpoints — Summary

Method	Type	Description
createBucket	update	Create a bucket (paid)
listBuckets	query	List the current user's buckets
getBucketStats	query	Usage, expiry, and file count
getRenewQuote	query	Renewal price
renewBucket	update	Extend the bucket by thirty days
beginFileUpload	update	Start a chunked upload; returns uploadId
uploadFileChunk	update	Send one chunk
completeFileUpload	update	Finalize and persist the file
uploadFile	update	Single-call upload, 700 KB or less
deleteFile	update	Remove a file
downloadFile	query	Decrypted blob, authenticated
listFiles	query	Paginated file list
getPublicFileUrl	query	CDN URL string
createApiKey	update	Create a scoped key
listApiKeys	query	List keys, secrets not included
revokeApiKey	update	Revoke a key

13. Deploy Checklist

Step	Command	Notes
Backend tests	npm run ci backend:test	Must pass 46 of 46
Frontend build	npm run ci frontend:build	Type-check plus static export
Merge to main	PR merged to main	Vercel deploys the frontend automatically
Backend deploy	npm run ci backend:deploy	Required — the chunk API lives on the canister
Verify upload	Upload a 5 MB .php file on icpay.app	Should succeed once both deploys are complete

Merging to main alone does not upgrade the canister. Without running backend:deploy, chunked uploads will fail, because the mainnet canister will lack the beginFileUpload, uploadFileChunk, and completeFileUpload methods.

14. Troubleshooting

Error	Cause	Fix
Message byte size larger than max allowed 2097152	Whole file sent in a single IC call	Deploy frontend with storeFile() and backend with the chunk API
Upload failed (generic)	Old canister without chunk methods	Run backend:deploy
File too large — max 10 MB	File exceeds the product cap	Split the file, or increase the cap in Config and redeploy
Chunk too large	Single chunk exceeds 700 KB	Check UPLOAD_CHUNK_BYTES is synchronized between frontend and backend
Rate limit	More than 40 upload calls per minute	Wait 60 seconds, then retry
Bucket expired	The 30-day period has ended	Renew the bucket from the detail page

15. Full System Diagram

The diagram below summarizes the complete system, from the Vercel-hosted frontend through the backend canister to the ICP Ledger, stable memory, and the public raw.icp0.io endpoint.

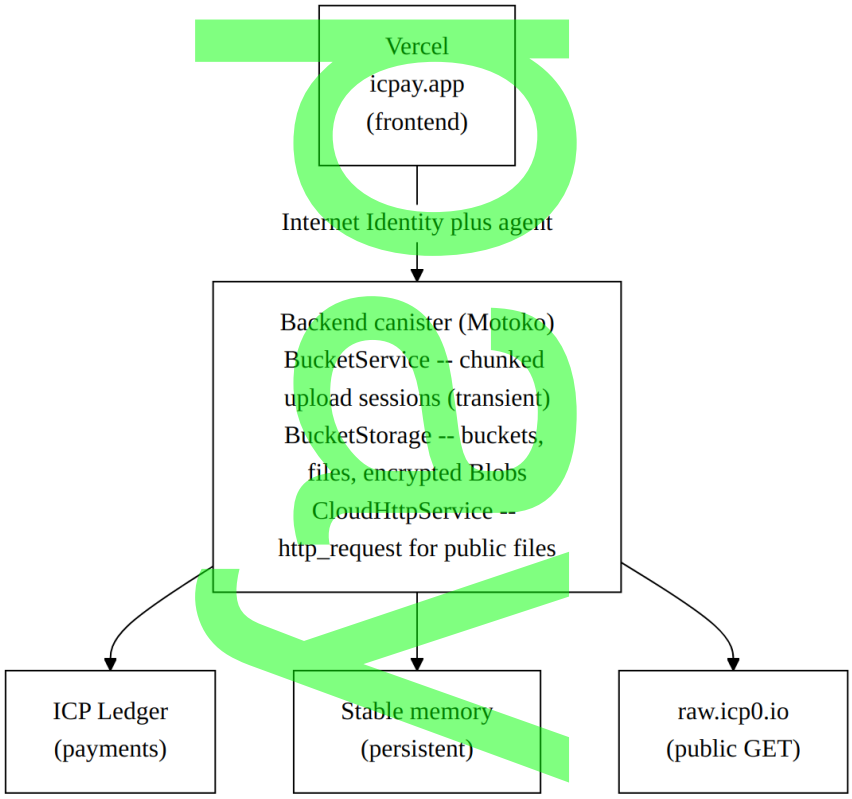

Figure 4 — End-to-end system overview

This document describes the Bucket module as shipped in PR #50. For operational commands, refer to docs/command/README.md.